Université Paris Nanterre

Licence MIASHS première année

Rapport de projet informatique

Quant Terminal : Simulateur d'impact d'événements économiques sur un portefeuille financier



Projet réalisé du 3 avril 2026 au 23 mai 2026

Dépôt GitHub : `https://github.com/00quentin/projet-info-ia-L1-QG-LD-AR-ES`

Membres du groupe

Quentin Geldreich — 45009207
Lucas Doazan — 45004502
Evan Saadi — 45019089
Alex Ruimy — 45015710

Remerciements

Nous tenons à remercier notre professeur pour ce cours d'informatique qui nous a permis de découvrir la programmation de sites web et de mener à bien ce projet.

Nous remercions également les équipes derrière les outils que nous avons utilisés : Streamlit, OpenAI et Yahoo Finance, sans lesquels ce projet n'aurait pas été possible.

Enfin, nous remercions l'intelligence artificielle Claude, développée par Anthropic, qui a joué un rôle central dans le développement de ce projet. Nous tenons à être transparents là-dessus : Claude a été bien plus qu'un simple correcteur de syntaxe. C'est avec lui qu'on a débogué les simulations irréalistes, géré les bugs de sidebar, et construit l'architecture du projet. Ce rapport parle d'un projet de programmation assistée par IA, et c'est vraiment ce que c'était.

Table des matières

1 Introduction **4**

2 Environnement de travail **5**
2.1 Outils de développement 5
2.2 Collaboration et versionnage 5
2.3 Déploiement 5
2.4 Organisation du groupe 5

3 Description du projet et objectifs **6**
3.1 Cahier des charges initial 6
3.2 Architecture du projet 6
3.3 Fonctionnement général 6
3.4 Le mouvement brownien géométrique 7

4 Bibliothèques, Outils et technologies **8**
4.1 Note sur Streamlit 8
4.2 Note sur la validation Pydantic 8

5 Travail réalisé **9**
5.1 Fonctionnalités réalisées 9
5.2 Fonctionnalités non réalisées 9
5.3 Répartition réelle du travail 10

6 Difficultés rencontrées **11**
6.1 Les limitations de Streamlit 11
6.2 La fiabilité des simulations financières 11
6.3 Le déploiement sur Streamlit Cloud 11
6.4 Le comportement du chatbot 12

7 Bilan **13**
7.1 Conclusion 13
7.2 Perspectives 13

8 Bibliographie **14**

9 Webographie **15**

A Cahier des charges **16**

B Exemple d'exécution du projet **16**

C Manuel utilisateur **32**

1 Introduction

Contexte

Aujourd'hui, les marchés financiers sont au cœur de l'économie mondiale. Les crises, les décisions des banques centrales ou les avancées technologiques ont des répercussions directes sur la valeur des actifs financiers. Pourtant, comprendre et anticiper ces impacts reste difficile pour quelqu'un qui n'est pas expert en finance.

En tant qu'étudiants en première année de double licence MIASHS et Économie-Gestion à l'Université Paris Nanterre, nous avons un intérêt commun pour l'investissement et la finance de marché. Ce projet est né de l'envie de combiner nos deux domaines d'étude : les mathématiques et l'informatique d'un côté, l'économie et la finance de l'autre.

Problématique

Comment permettre à un étudiant ou à un particulier, sans connaissances avancées en finance, de simuler l'impact d'un événement économique sur un portefeuille d'investissement de manière réaliste et pédagogique ?

Notre solution : Quant Terminal

Quant Terminal est une application web interactive développée en Python avec le framework Streamlit. L'utilisateur décrit un événement économique en langage naturel (une hausse des taux directeurs, une crise technologique, un choc géopolitique) et observe l'impact simulé sur un portefeuille financier personnalisé.

L'application s'appuie sur l'intelligence artificielle (OpenAI GPT) pour analyser le scénario et estimer les chocs sur chaque actif, sur Yahoo Finance pour récupérer les vrais prix de marché en temps réel, et sur des modèles mathématiques (mouvement brownien géométrique, Monte-Carlo) pour simuler l'évolution du portefeuille jour après jour.

Plan du rapport

Ce rapport présente d'abord notre environnement de travail et les outils utilisés, puis l'architecture et le fonctionnement du projet avec les modèles mathématiques sous-jacents. Nous détaillons ensuite les fonctionnalités réalisées, les difficultés rencontrées, et nous terminons par un bilan de cette expérience.

2 Environnement de travail

2.1 Outils de développement

Nous avons développé le projet en Python 3.10 sous Visual Studio Code. La gestion des dépendances se fait via un fichier `requirements.txt`, et la clé API OpenAI est stockée dans un fichier `.env` pour des raisons de sécurité. Ce fichier n'est jamais envoyé sur GitHub.

L'outil qui a le plus accompagné notre développement est **Claude Code** (Anthropic), un assistant IA intégré directement dans Visual Studio Code. Nous avons ouvert notre dossier de projet entier dans Claude Code, ce qui lui permettait d'avoir une vision globale du code. En pratique, on décrivait une fonctionnalité en français, Claude proposait un code, on testait, ça ne marchait pas toujours, on corrigeait ensemble. C'est ce cycle qui a structuré tout notre développement.

2.2 Collaboration et versionnage

Nous avons utilisé **Git** et **GitHub** pour gérer les versions du projet. Le dépôt est public et accessible à l'adresse suivante :

`https://github.com/00quentin/projet-info-ia-L1-QG-LD-AR-ES`

Les commits étaient principalement réalisés par les membres chargés du développement. Les décisions importantes étaient discutées collectivement par les quatre membres.

2.3 Déploiement

L'application est déployée en ligne via **Streamlit Cloud**, accessible gratuitement depuis n'importe quel navigateur. Un problème de mise en veille automatique après quelques minutes d'inactivité a été résolu grâce à **Uptime Robot**, un service qui envoie une requête au site toutes les cinq minutes pour le maintenir actif.

2.4 Organisation du groupe

Le groupe est composé de quatre membres. La collaboration s'est organisée en deux phases : plusieurs séances en présentiel durant les TD pour définir les objectifs et répartir le travail, puis une longue phase de développement à distance, avec une réunion finale pour tout mettre en forme.

3 Description du projet et objectifs

3.1 Cahier des charges initial

L'idée de départ était simple : permettre à un utilisateur de saisir un événement économique et d'observer son impact sur un portefeuille. En cours de développement, nous avons progressivement enrichi le projet avec de nombreuses fonctionnalités supplémentaires qui n'étaient pas prévues au départ.

Les fonctionnalités prévues dès le début étaient :

— Saisie d'un événement économique en langage naturel
— Analyse de l'événement par intelligence artificielle
— Simulation de l'impact sur un portefeuille personnalisable
— Visualisation des résultats sous forme de graphiques interactifs
— Affichage de métriques financières (rendement, risque)

3.2 Architecture du projet

Le projet est organisé en modules distincts pour séparer clairement la logique métier de l'interface utilisateur. La règle principale que nous avons suivie : le dossier `core/` ne doit jamais importer Streamlit. Cela permet de tester la logique indépendamment de l'interface, et d'éviter que tout le projet s'effondre quand Streamlit change de version.

Fichier / Dossier	**Rôle**
`app.py`	Point d'entrée principal, orchestre les onglets et la navigation
`core/`	Logique métier pure (calculs, métriques, historique) sans import Streamlit
`components/`	Composants d'interface réutilisables (sidebar, graphiques, notifications)
`views/`	Un fichier par onglet (dashboard, portefeuille, comparaison, etc.)
`handlers.py`	Orchestration des lancements de simulation et de backtest
`ia_bot.py`	Appels à l'API OpenAI avec validation des réponses
`market_data.py`	Récupération des prix via Yahoo Finance avec mise en cache
`simulation.py`	Moteur de simulation (probabiliste, historique, Monte-Carlo)
`schemas.py`	Validation des réponses de l'IA via Pydantic
`config.py`	Toutes les constantes du projet (actifs, profils, bornes)

Table 1 – Architecture modulaire de Quant Terminal

3.3 Fonctionnement général

Voici les étapes qu'un utilisateur suit pour réaliser une simulation :

1. Choisir le mode : *Simulation prospective* (scénario fictif) ou *Backtest historique* (crise réelle passée)
2. Décrire un événement économique en langage naturel dans la sidebar
3. Sélectionner les actifs à inclure (jusqu'à 23 actifs disponibles)
4. Définir le capital de départ et le profil d'investisseur
5. Cliquer sur *Lancer la simulation*
6. L'IA analyse le scénario et estime les chocs sur chaque actif, validés par Pydantic pour éviter les valeurs aberrantes
7. Le moteur calcule l'évolution du portefeuille jour après jour
8. Les résultats s'affichent : graphiques, métriques, analyse IA, alertes

3.4 Le mouvement brownien géométrique

Le moteur de simulation repose sur le **mouvement brownien géométrique** (GBM), un modèle standard en finance quantitative, utilisé notamment dans la formule de Black-Scholes pour la valorisation des options. L'équation différentielle stochastique qui le décrit est :

$$dS_t = \mu\, S_t\, dt + \sigma\, S_t\, dW_t \tag{1}$$

où S_t est la valeur de l'actif au temps t, μ le drift (tendance directionnelle imposée par le choc estimé par l'IA), σ la volatilité historique annualisée récupérée via Yahoo Finance sur 12 mois, et dW_t un processus de Wiener représentant le bruit aléatoire du marché.

En pratique, à chaque jour de cotation simulé, la variation de chaque actif est calculée via la solution exacte du GBM :

$$S_{t+1} = S_t \cdot \exp\left[\left(\mu - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}\cdot\varepsilon\right] \tag{2}$$

où $\varepsilon \sim \mathcal{N}(0,1)$ est un nombre aléatoire tiré d'une loi normale standard (généré via NumPy). Le terme $-\sigma^2/2$ est une correction d'Itô qui compense le biais introduit par la nature multiplicative du processus.

Nous avons choisi ce modèle pour trois raisons : il garantit que les prix restent toujours positifs (contrairement à un modèle additif), il est calibrable directement à partir des données Yahoo Finance, et c'est le modèle de référence académique en finance stochastique.

Mode Monte-Carlo : en activant ce mode, le processus est répété 50 fois avec des tirages ε différents. On obtient ainsi une enveloppe de trajectoires possibles plutôt qu'une courbe unique. Les zones colorées sur les graphiques représentent les percentiles 5% et 95% de ces 50 simulations.

4 Bibliothèques, Outils et technologies

Bibliothèque	Rôle dans le projet	Pourquoi ce choix
Streamlit	Framework principal pour l'interface web	Simple en Python pur, déploiement gratuit
OpenAI API	Analyse des scénarios par IA	GPT comprend le langage naturel en contexte financier
yfinance	Prix réels via Yahoo Finance	Gratuit, fiable, supporte tous nos actifs
Plotly	Graphiques interactifs	Rendu professionnel, natif dans Streamlit
Pydantic	Validation des réponses de l'IA	Évite les valeurs aberrantes type Or +1 000 000%
NumPy	Calculs du GBM et Monte-Carlo	Référence pour les maths scientifiques en Python
Pandas	Séries temporelles journalières	Facilite la manipulation des données de marché
ReportLab	Génération du rapport PDF	Création de PDF directement en Python
python-dotenv	Gestion de la clé API	Sécurise la clé OpenAI hors du code source

Table 2 – Bibliothèques utilisées dans Quant Terminal

4.1 Note sur Streamlit

Streamlit est une bibliothèque Python qui transforme automatiquement chaque widget Python (bouton, slider, graphique) en composant web interactif, sans avoir besoin d'écrire du HTML ou du JavaScript. C'est ce qui nous a permis, avec notre niveau de L1, de créer une vraie application web.

Sa limite principale : Streamlit recrée entièrement la page à chaque interaction utilisateur. Ça simplifie beaucoup de choses, mais ça rend certains comportements d'interface très difficiles à contrôler, comme on l'a découvert avec le bouton de la sidebar.

4.2 Note sur la validation Pydantic

Pydantic est une bibliothèque de validation de données en Python. Dans notre projet, elle sert à vérifier que les chocs estimés par l'IA sont dans des plages réalistes avant d'être injectés dans le GBM. Par exemple, le schéma de validation pour le S&P 500 impose une borne entre -55% et +45%, calibrée sur les pires crises historiques (2008 : -57% sur 18 mois). Si l'IA dépasse ces bornes, Pydantic lève une erreur, la valeur est corrigée et un avertissement est affiché.

5 Travail réalisé

5.1 Fonctionnalités réalisées

Simulation prospective : saisie d'un événement fictif, analyse IA, simulation GBM sur 1 à 3 scénarios comparables simultanément
Backtest historique : rejouer de vraies crises (COVID-19, 2008, Choc pétrolier 1973, etc.) avec les données réelles Yahoo Finance
Mode comparaison : graphique d'évolution comparée et tableau de métriques côte à côte pour 2 ou 3 scénarios
Mode Monte-Carlo : 50 simulations GBM parallèles avec enveloppe probabiliste (percentiles 5-95%)
Calibration historique IA : l'IA s'inspire des amplitudes réelles de crises passées pour calibrer ses estimations de chocs
Portefeuille personnalisable : capital, profil d'investisseur (Prudent / Équilibré / Agressif / Personnalisé), allocation manuelle
Actifs personnalisés : ajout de n'importe quel actif via son ticker Yahoo Finance (TSLA, BTC-USD, etc.)
Prix en temps réel : vrais prix Yahoo Finance avec cache d'une heure et bouton de rafraîchissement manuel
Métriques institutionnelles : Volatilité annualisée, Sharpe Ratio, Max Drawdown, VaR 95% avec alertes automatiques
Historique des simulations : sauvegarde automatique des 10 dernières simulations en JSON, persistante entre les sessions
Export CSV : données journalières brutes par actif
Export PDF : rapport complet rédigé par l'IA avec analyse, métriques et recommandations
Chatbot IA : analyste financier conversationnel accessible via une bulle flottante
Académie pédagogique : 7 modules sur la finance de marché (lecture des données, macro-économie, cas historiques, stratégies, etc.)
Mode sombre / clair : interface adaptable
Partage par URL : paramètres de simulation encodés dans l'URL pour partager une configuration
Ticker en direct : S&P 500, VIX, Or, Pétrole, BTC, EUR/USD mis à jour en temps réel

5.2 Fonctionnalités non réalisées

Certaines idées ont été abandonnées en cours de route. Nous voulions une interface visuellement plus proche des plateformes fintech professionnelles, mais les contraintes de Streamlit nous en ont empêchés. Chaque effet visuel avancé nécessitait des contournements CSS complexes qu'on n'avait pas anticipés.

Des fonctionnalités plus ambitieuses comme l'authentification utilisateur ou une base de données pour l'historique ont été écartées : elles dépassaient le cadre d'un projet de première année et le temps disponible.

5.3 Répartition réelle du travail

Membre	**Tâches réalisées**
Quentin Geldreich	Développement principal : architecture générale, simulation GBM, intégration OpenAI et Yahoo Finance, interface Streamlit, déploiement en ligne
Lucas Doazan	Co-développement, tests réguliers, suggestions de fonctionnalités, participation aux décisions techniques et à la revue de code
Evan Saadi	Rédaction du rapport, recherches sur les concepts financiers (GBM, métriques de risque), tests fonctionnels de l'application, retours utilisateur
Alex Ruimy	Rédaction du rapport, tests de l'application sur différents scénarios, retours sur l'expérience utilisateur, suggestions d'amélioration de l'interface

TABLE 3 – Répartition réelle du travail au sein du groupe

Toutes les décisions importantes (fonctionnalités à ajouter, scénarios à tester, orientation du projet) étaient discutées collectivement. Les membres chargés du rapport ont aussi joué un rôle de testeurs réguliers, en signalant les bugs et en vérifiant que les résultats de simulation semblaient cohérents.

6 Difficultés rencontrées

6.1 Les limitations de Streamlit

La principale difficulté technique du projet a été de travailler avec Streamlit sur des interfaces un peu avancées. Le framework est excellent pour démarrer rapidement, mais il montre vite ses limites quand on veut personnaliser le comportement de certains composants.

Le cas le plus frustrant a été le bouton d'ouverture et de fermeture de la sidebar. Ce bouton fonctionnait parfaitement au départ, puis disparaissait définitivement après le premier clic de fermeture, rendant la sidebar totalement inaccessible. On a mis plusieurs jours à comprendre pourquoi : Streamlit réécrit entièrement le DOM (la structure de la page web) en JavaScript à chaque interaction, écrasant systématiquement nos modifications CSS. On a essayé plusieurs approches : injecter du JavaScript pour intercepter le clic, cibler le bouton par différents sélecteurs CSS, modifier le comportement via les query params. Rien ne tenait sur la durée. La solution finale a été de forcer la sidebar à rester toujours visible via du CSS `!important` et de cacher le bouton de fermeture. Ce n'est pas élégant, mais c'est la seule approche stable qu'on ait trouvée.

En fait, chaque effet visuel un peu avancé nécessitait des contournements qu'on n'avait pas anticipés au départ. Le design qu'on imaginait au début aurait probablement nécessité de travailler avec React ou une autre technologie.

6.2 La fiabilité des simulations financières

Lors des premières simulations, l'IA renvoyait des valeurs complètement irréalistes. L'exemple le plus marquant : dans l'une de nos premières versions, l'Or affichait une variation de **+1 222 891%** lors d'un scénario de crise. Ce genre de résultat rendait l'application absurde et inutilisable.

Comprendre pourquoi ça arrivait a pris du temps. Le problème venait du fait qu'on demandait à l'IA d'estimer des pourcentages de variation sans lui donner de cadre de référence. Sans contrainte, GPT optimisait pour la "dramatisation" du scénario plutôt que pour le réalisme. On a résolu ça en deux étapes : d'abord en précisant dans le prompt les ordres de grandeur attendus, puis en ajoutant une validation Pydantic avec des bornes calibrées sur les vraies crises historiques. Depuis, les simulations produisent des résultats cohérents.

6.3 Le déploiement sur Streamlit Cloud

Le déploiement a posé plusieurs problèmes qu'on n'avait pas anticipés. Le principal : l'application se mettait en veille automatiquement après quelques minutes d'inactivité, obligeant les visiteurs à attendre 30 secondes à chaque rechargement. On a contourné ça avec Uptime Robot, un service gratuit qui envoie une requête au site toutes les cinq minutes.

La configuration de la clé API OpenAI a aussi nécessité une manipulation spécifique : sur Streamlit Cloud, le fichier `.env` local n'existe pas. Il faut renseigner les secrets directement dans l'interface de déploiement.

6.4 Le comportement du chatbot

Au départ, le chatbot répondait à n'importe quelle question sans se limiter à la finance. On a dû modifier le prompt système pour le recadrer sur son domaine. Ce travail de "prompt engineering" (la manière dont on formule les instructions données à un modèle de langage) a été plus complexe qu'attendu : trop restrictif, le chatbot refusait des questions légitimes ; trop permissif, il dérivait vers des sujets sans rapport. On a fini par trouver un équilibre qui fonctionne bien.

7 Bilan

7.1 Conclusion

Honnêtement, on ne pensait pas arriver à un résultat comme celui-là quand on a commencé. L'idée de départ était simple : saisir un événement, voir l'impact sur un portefeuille. Un mois et demi plus tard on a une application déployée en ligne avec 17 fonctionnalités, des graphiques interactifs, un modèle Monte-Carlo et un chatbot IA intégré. C'est difficile de ne pas en être fiers.

On a appris des tas de choses sans vraiment s'en rendre compte. Au détour d'un bug à corriger on comprenait comment fonctionne le cache en Python ; en essayant de rendre les simulations réalistes on s'est retrouvés à lire des articles sur les crises de 1973 et sur le modèle de Black-Scholes ; en déployant sur Streamlit Cloud on a découvert ce qu'est une variable d'environnement et pourquoi on ne commit jamais une clé API sur GitHub. C'est ce genre d'apprentissage concret qu'on n'aurait pas eu en restant sur des exercices académiques.

Sur l'IA : Claude a été un vrai partenaire de développement, pas juste un auto-correcteur. Ce n'est pas "l'IA a fait le projet à notre place". C'est plus proche de : l'IA nous a permis d'aller bien au-delà de notre niveau technique actuel. Sans elle, on aurait probablement produit un script Python basique. Avec elle, on a produit une vraie application. Mais sans notre direction, nos tests et nos corrections, et il y en a eu beaucoup, Claude aurait généré des simulations avec l'or à +1 222 891% et une sidebar qui disparaît au premier clic.

Si on devait refaire ce projet, on passerait davantage de temps à planifier l'architecture avant de commencer à coder. On a beaucoup ajouté de fonctionnalités au fil de l'eau, ce qui a parfois créé des incohérences qu'on a dû corriger ensuite. On choisirait aussi probablement une technologie plus flexible que Streamlit pour le design.

7.2 Perspectives

On a envie de continuer ce projet après le cours. Ce qui manque le plus au modèle actuel, c'est la corrélation entre actifs : aujourd'hui chaque actif est simulé indépendamment, alors que dans la réalité l'or et le dollar bougent souvent en sens inverse, tout comme les obligations par rapport aux actions. Ce serait la prochaine étape logique côté maths. On aimerait aussi remplacer la sauvegarde en JSON par une vraie base de données, avec un système d'authentification pour que chaque utilisateur garde son propre historique. Et si on recommençait depuis zéro, on choisirait probablement une technologie plus flexible que Streamlit pour l'interface.

8 Bibliographie

[BLACK73] F. Black, M. Scholes, *The Pricing of Options and Corporate Liabilities*, Journal of Political Economy, Vol. 81, No. 3, 1973

[HULL] J. Hull, *Options, Futures, and Other Derivatives*, 11e édition, Pearson, 2022

[ITO] K. Itô, *On Stochastic Differential Equations*, Memoirs of the American Mathematical Society, 1951

9 Webographie

[ST] Documentation officielle Streamlit, https://docs.streamlit.io

[OAI] Documentation OpenAI API, https://platform.openai.com/docs

[YF] yfinance sur PyPI, https://pypi.org/project/yfinance

[PD] Documentation Pydantic v2, https://docs.pydantic.dev

[PL] Documentation Plotly Python, https://plotly.com/python

[GBM] Wikipédia, Mouvement brownien géométrique, https://fr.wikipedia.org/wiki/Mouvement_brownien_g%C3%A9om%C3%A9trique

[GH] Dépôt GitHub du projet, https://github.com/00quentin/projet-info-ia-L1-QG-LD-AR-ES

[UR] Uptime Robot, monitoring de disponibilité, https://uptimerobot.com

Annexe A : Cahier des charges

Le cahier des charges initial comprenait les fonctionnalités suivantes :
— Saisie d'un événement économique en langage naturel
— Analyse de l'événement par intelligence artificielle (OpenAI GPT)
— Sélection des actifs et du profil d'investisseur
— Simulation de l'évolution d'un portefeuille sur un horizon défini
— Affichage de graphiques interactifs et de métriques financières

Les fonctionnalités suivantes ont été ajoutées hors cahier des charges initial, au fil du développement : mode backtest historique, comparaison de scénarios, Monte-Carlo, calibration historique IA, export CSV/PDF, chatbot IA, académie pédagogique, mode sombre, partage par URL, ticker en direct, actifs personnalisés via ticker Yahoo.

Annexe B : Exemple d'exécution du projet

Cette annexe présente une session complète d'utilisation de Quant Terminal. Nous avons simulé deux scénarios économiques opposés sur le même portefeuille de 15 000€ avec un profil Agressif, 20 actifs sélectionnés, un horizon de 130 jours de cotation et le mode Monte-Carlo activé (50 simulations parallèles).

Scénario A (Stagflation européenne) : la BCE monte ses taux à 5,5% malgré une récession, l'euro chute, les marchés actions européens s'effondrent mais l'or et le pétrole résistent en valeur refuge.

Scénario B (Boom IA et semi-conducteurs asiatiques) : la Chine lève ses restrictions sur les semi-conducteurs, les valeurs tech explosent, les actifs risqués surperforment massivement.

Résultats : Scénario A perd **-10,81%** (13 379€), Scénario B gagne **+24,32%** (18 648€). L'écart de 35,12 points entre les deux trajectoires illustre la puissance de la fonctionnalité de comparaison.

Étape 1 : Page d'accueil et configuration

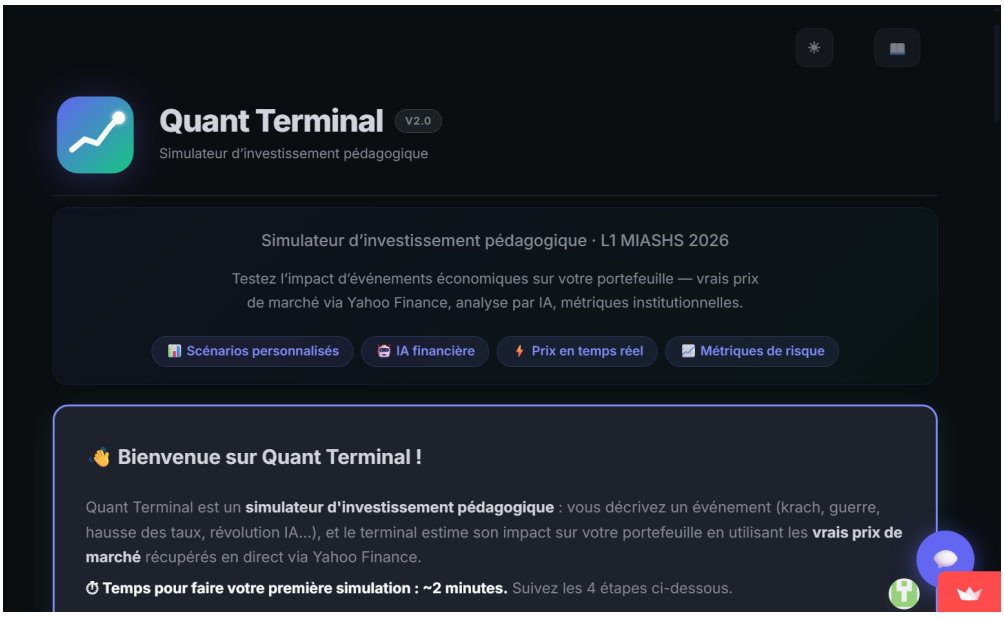


FIGURE 1 – Page d'accueil en mode sombre. Le ticker en haut affiche les prix en direct (S&P 500, VIX, Or, BTC) et un guide en 4 étapes explique comment démarrer

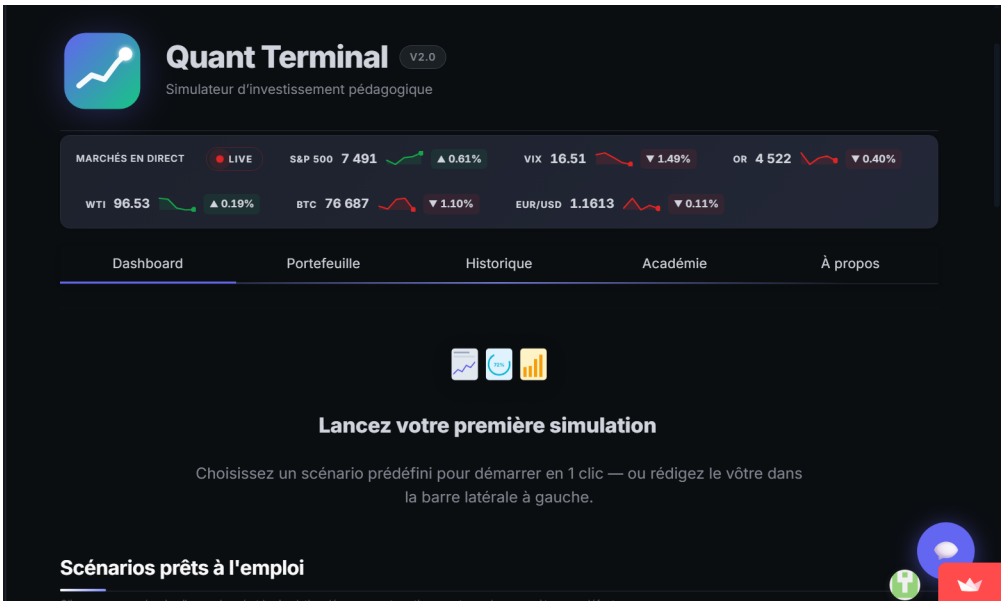


FIGURE 2 – Dashboard vide avant la première simulation, avec presets de scénarios pour aider l'utilisateur à démarrer

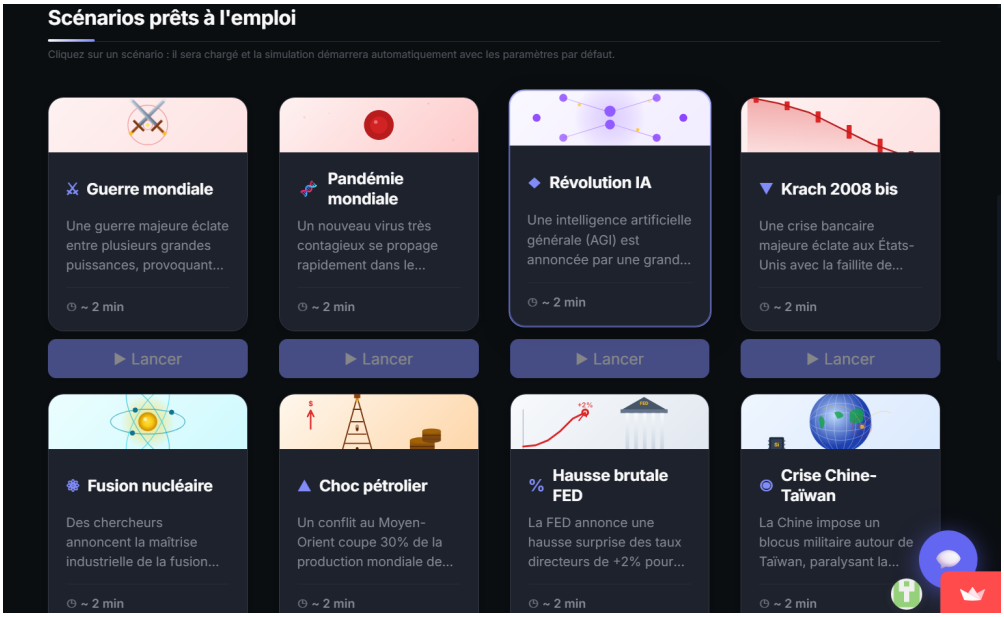


FIGURE 3 – Scénarios rapides prêts à l'emploi. Un clic charge la configuration complète sans avoir à remplir le formulaire manuellement

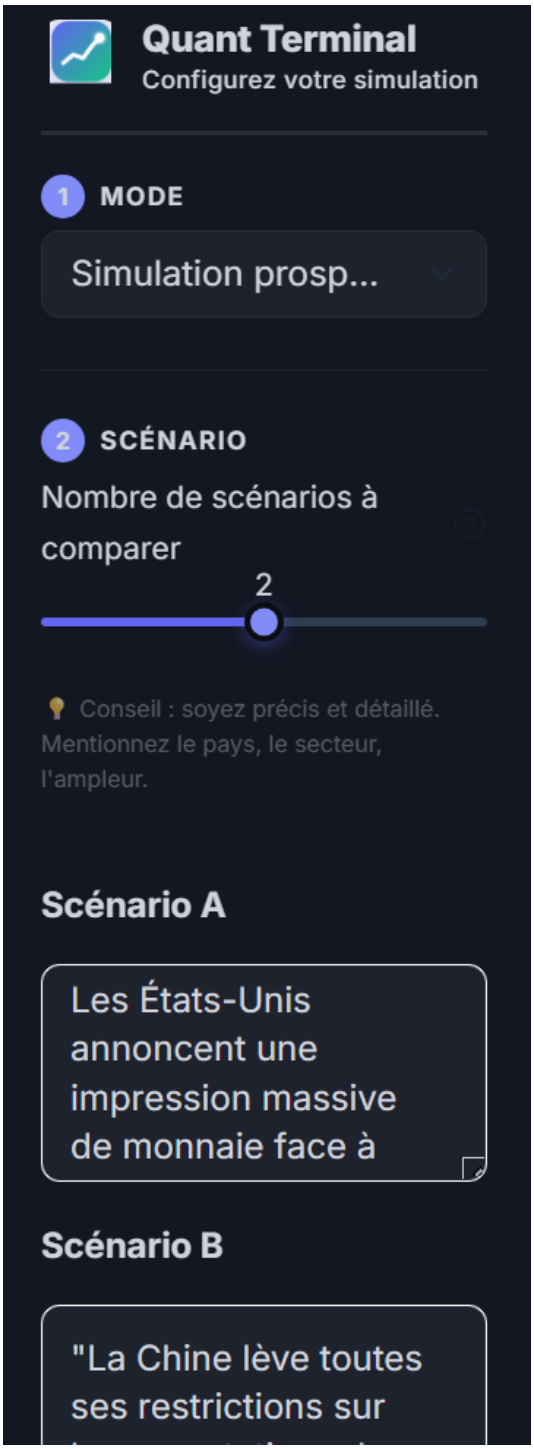


FIGURE 4 – Sidebar de configuration : mode Simulation prospective, 2 scénarios à comparer, saisie des événements A et B. Le conseil en jaune incite à détailler le scénario pour améliorer la qualité de l'analyse

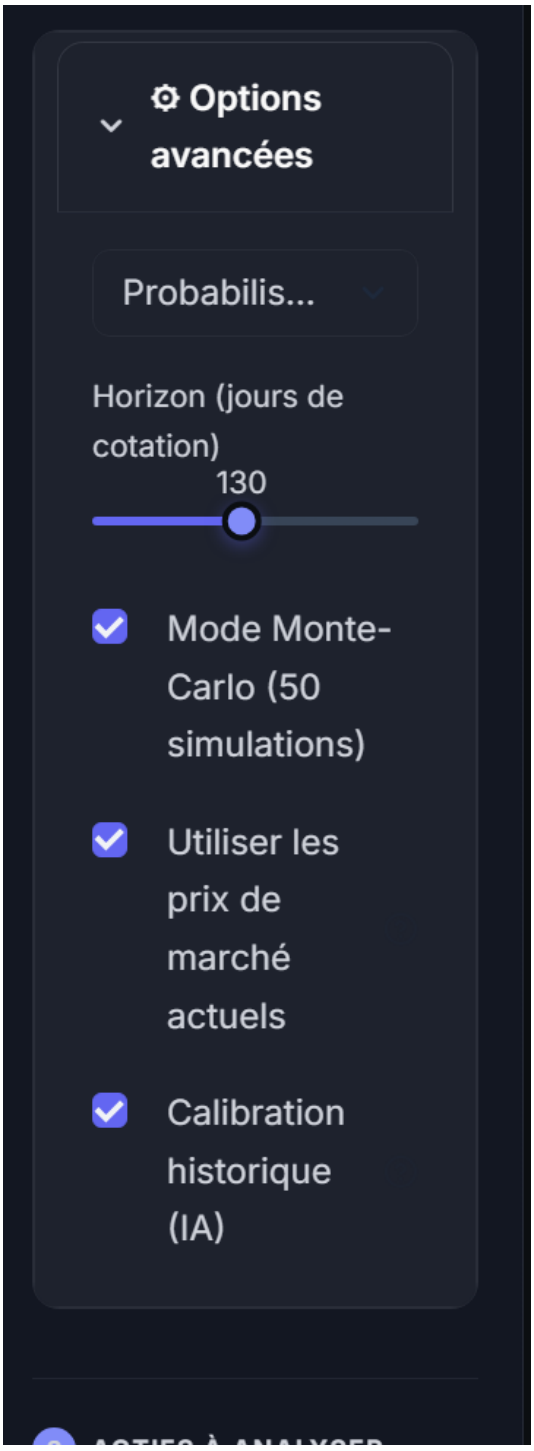


FIGURE 5 – Options avancées : Monte-Carlo activé (50 simulations), horizon de 130 jours, prix réels Yahoo Finance, calibration historique IA activée



FIGURE 6 – Sélection des actifs (20 sur 23) et paramètres du portefeuille : capital de 15 000€, profil Agressif

Étape 2 : Résultats du Dashboard

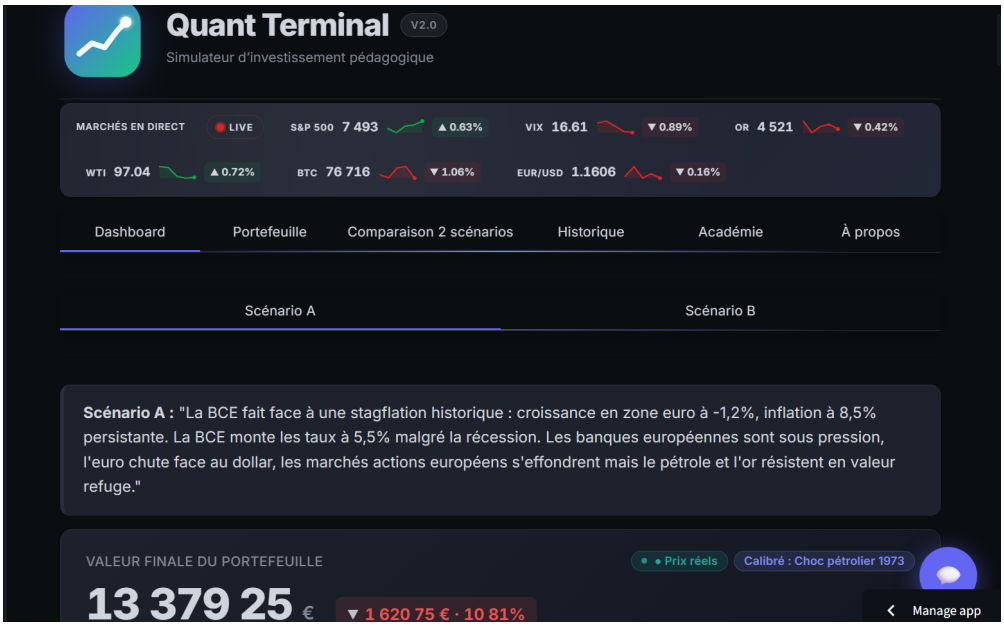


FIGURE 7 – Dashboard après simulation du Scénario A : valeur finale 13 379€ (-10,81%), calibré sur le Choc pétrolier de 1973

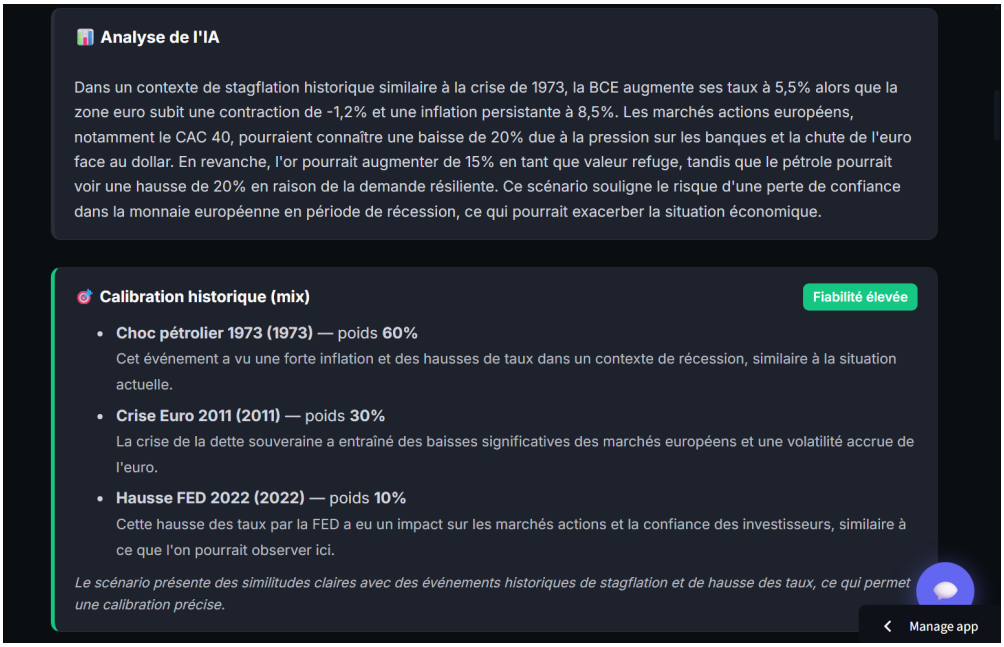


FIGURE 8 – Analyse macroéconomique rédigée par l'IA. GPT explique les mécanismes de transmission du scénario avec les estimations d'impact (inflation +8,52%, taux directeurs +5,53%)



FIGURE 9 – Évolution du portefeuille sur 130 jours comparée au S&P 500 (-16,18%) et au MSCI World (-16,44%). Notre portefeuille (-10,81%) surperforme les indices grâce à la présence d'or et de pétrole



FIGURE 10 – Métriques de risque : Volatilité 9,4% (FAIBLE), Sharpe -2,54 (MAUVAIS), Max Drawdown -11,6% (MODÉRÉ), VaR 95% -1,0%/jour. L'alerte rouge signale le rendement/risque défavorable

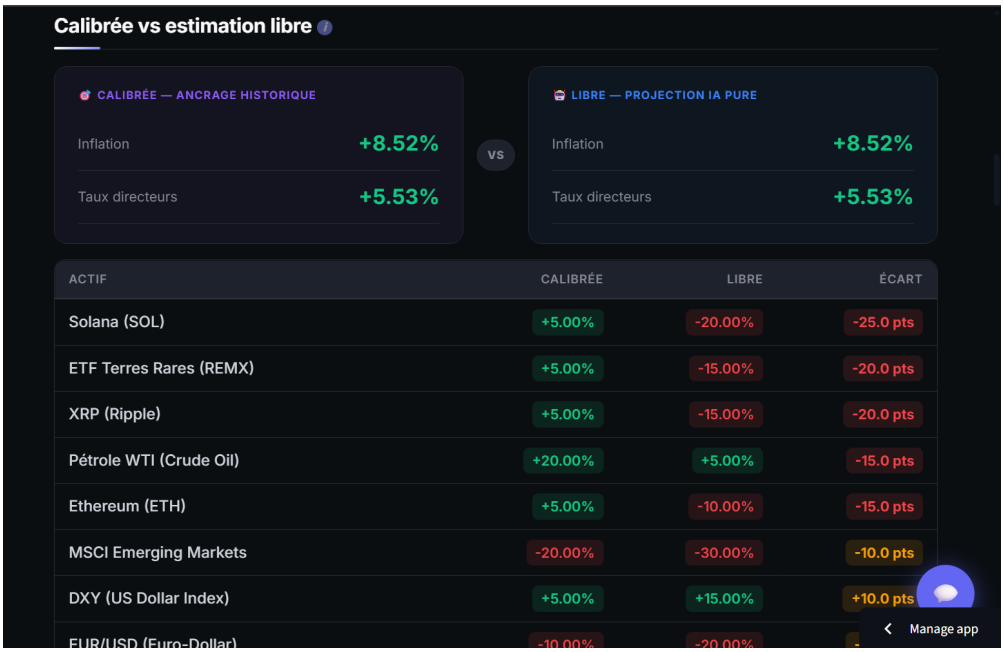


FIGURE 11 – Comparaison calibration historique vs projection IA libre par actif. Un écart supérieur à 15 points indique un scénario peu couvert par les données historiques

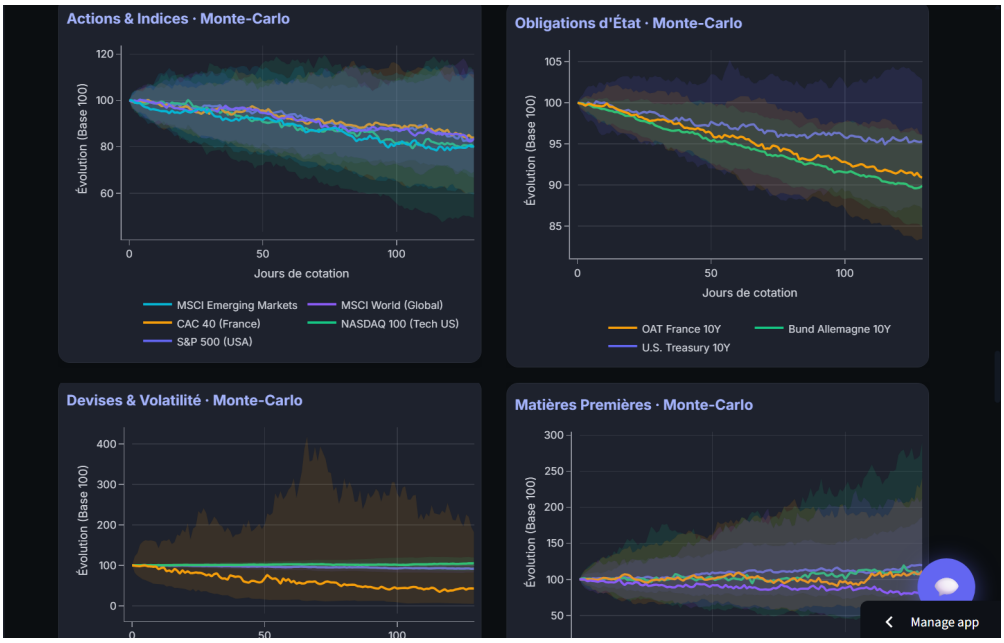


FIGURE 12 – Simulations Monte-Carlo par catégorie (50 trajectoires). Les zones colorées montrent l'enveloppe des scénarios possibles. Les actions ont une dispersion bien plus large que les obligations



FIGURE 13 – Monte-Carlo cryptomonnaies : Bitcoin, Ethereum, XRP et Solana sur 130 jours. La ligne pointillée blanche est la médiane des 50 simulations



FIGURE 14 – Performance par actif : Or +15%, Pétrole +12%, Défense +7% en vert, contre CAC 40 -16%, S&P 500 -16%, NASDAQ -20% en rouge

Étape 3 : Onglet Portefeuille



FIGURE 15 – Détail par actif : chaque carte affiche la valeur finale et la performance. Le contraste est net entre Pétrole (+11,85%) et NASDAQ (-20,33%)

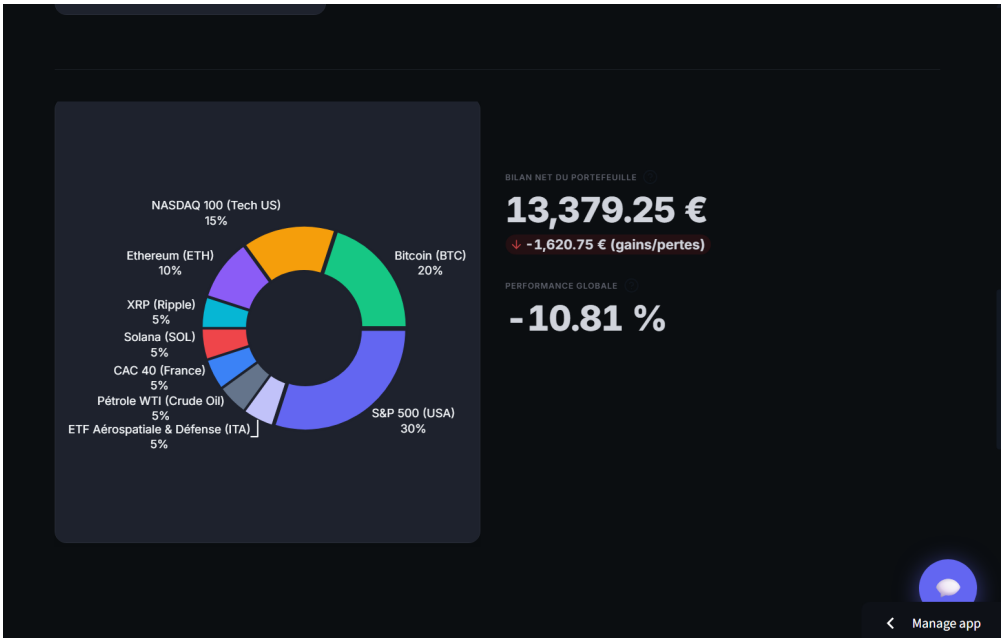


FIGURE 16 – Allocation (donut) et bilan net : S&P 500 à 30%, Bitcoin à 20%, NASDAQ à 15%. Résultat final : 13 379€ soit -1 620€ sur les 15 000€ initiaux

Étape 4 : Comparaison des deux scénarios



FIGURE 17 – Évolution comparée : Scénario B (vert, +24,32%) contre Scénario A (bleu, -10,81%). Les trajectoires divergent dès les premiers jours



FIGURE 18 – Métriques A vs B : Sharpe -2,54 contre +3,99, Max Drawdown -11,60% contre -3,50%. Le scénario B domine sur tous les indicateurs

Étape 5 : Export, Historique et fonctionnalités annexes

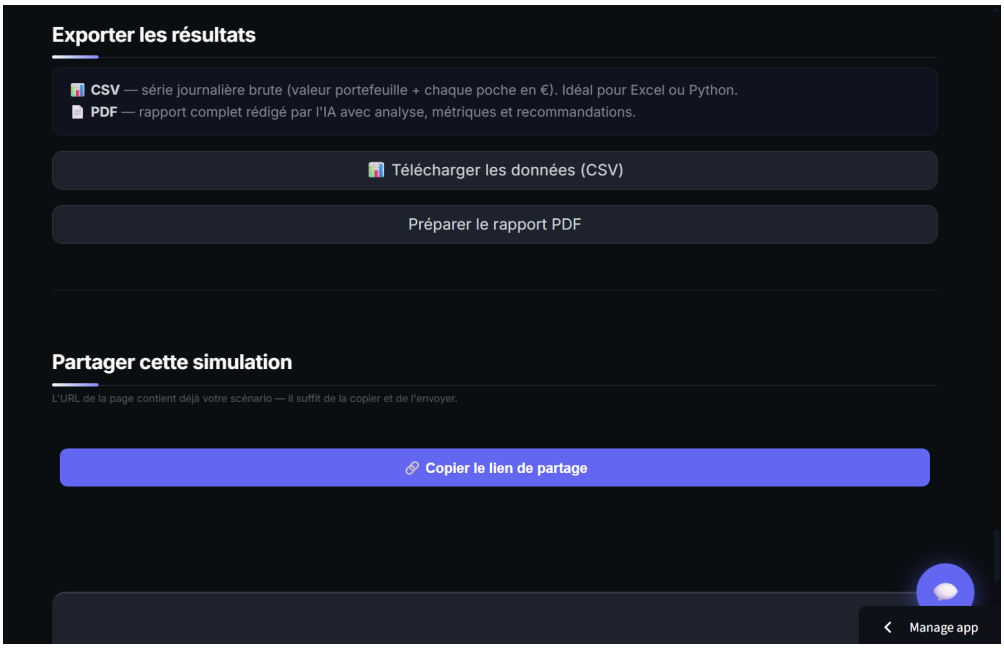


FIGURE 19 – Section export : données brutes en CSV, rapport PDF généré par l'IA, bouton de partage qui copie un lien URL avec tous les paramètres de simulation

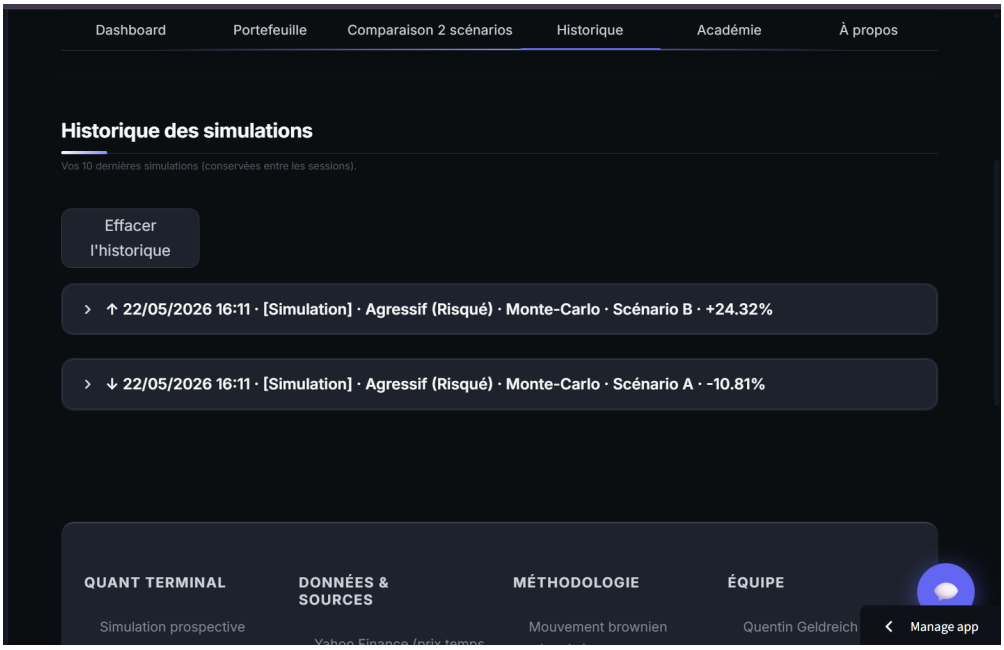


FIGURE 20 – Onglet Historique : les deux simulations sont sauvegardées avec date, profil et performance. L'historique persiste entre les sessions via un fichier JSON

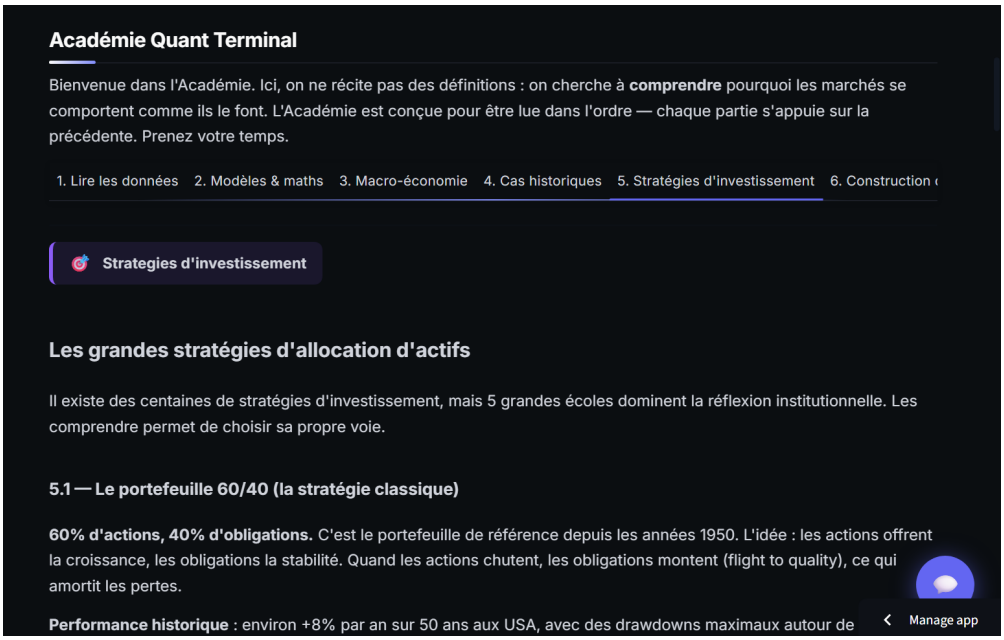


FIGURE 21 – Académie, module Stratégies d'investissement. Le portefeuille 60/40 (actions/obligations) est présenté comme référence institutionnelle depuis les années 1950

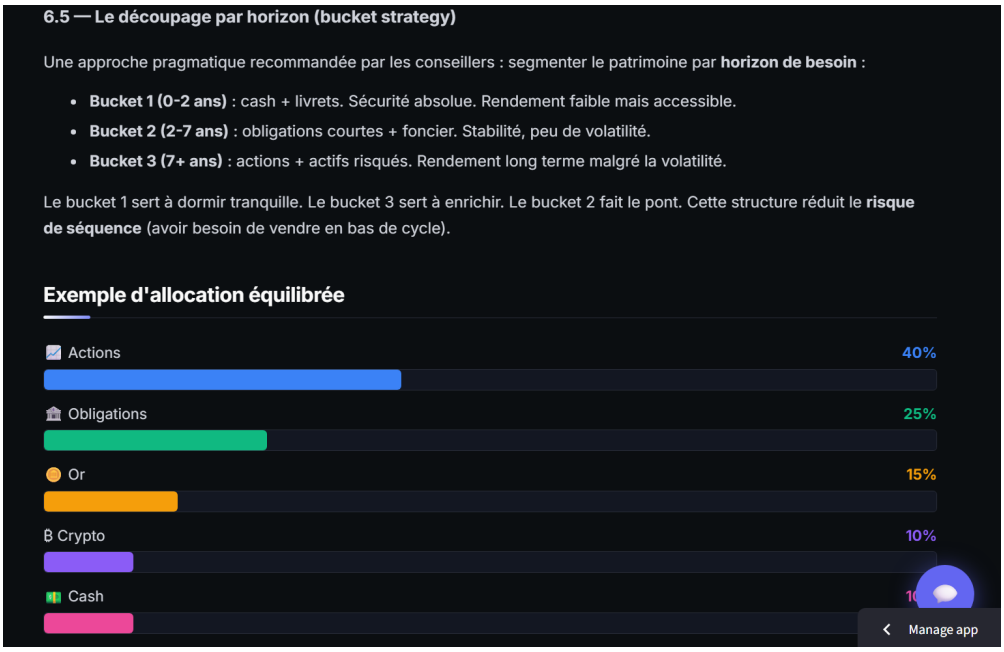


FIGURE 22 – Exemple d'allocation équilibrée : 40% actions, 25% obligations, 15% or, 10% crypto, 10% cash

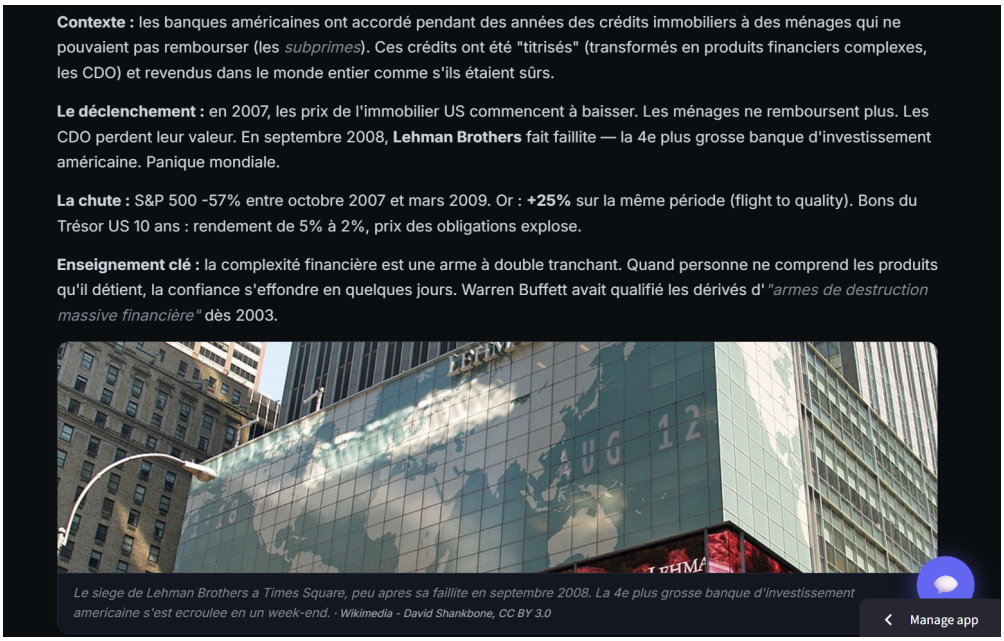


FIGURE 23 – Cas historique de la Crise de 2008 : contexte subprimes, déclenchement, chute de -57% sur le S&P 500 et enseignements



FIGURE 24 – Chatbot IA répondant à *Pourquoi l'Or monte-t-il en période de crise ?* Les questions fréquentes à gauche permettent de démarrer sans connaissances préalables

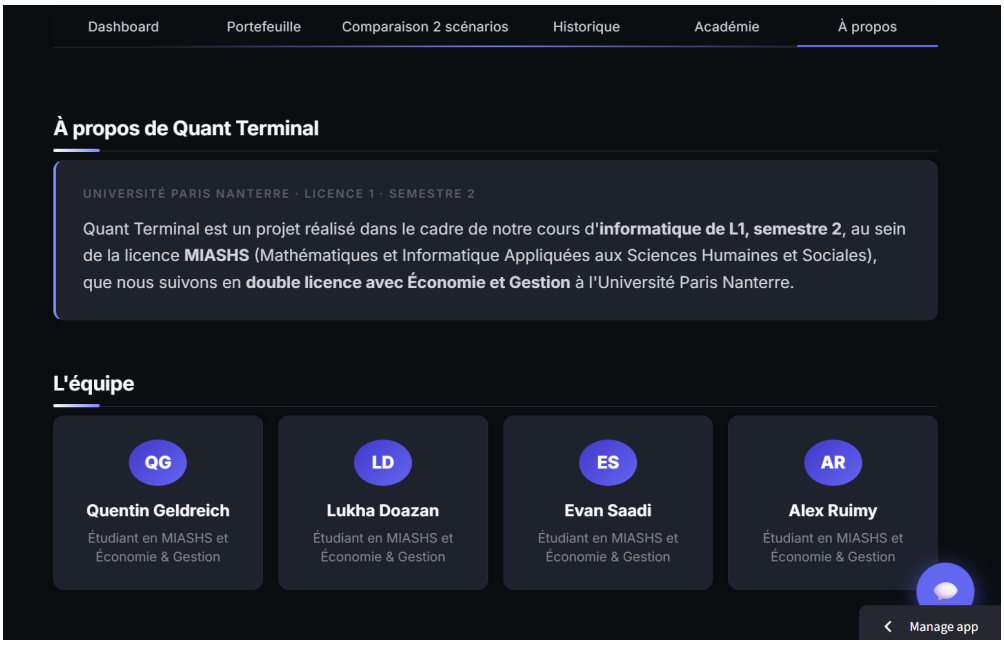


FIGURE 25 – Page À propos : présentation du projet et contexte académique (L1 MIASHS, double licence Économie-Gestion, Université Paris Nanterre)

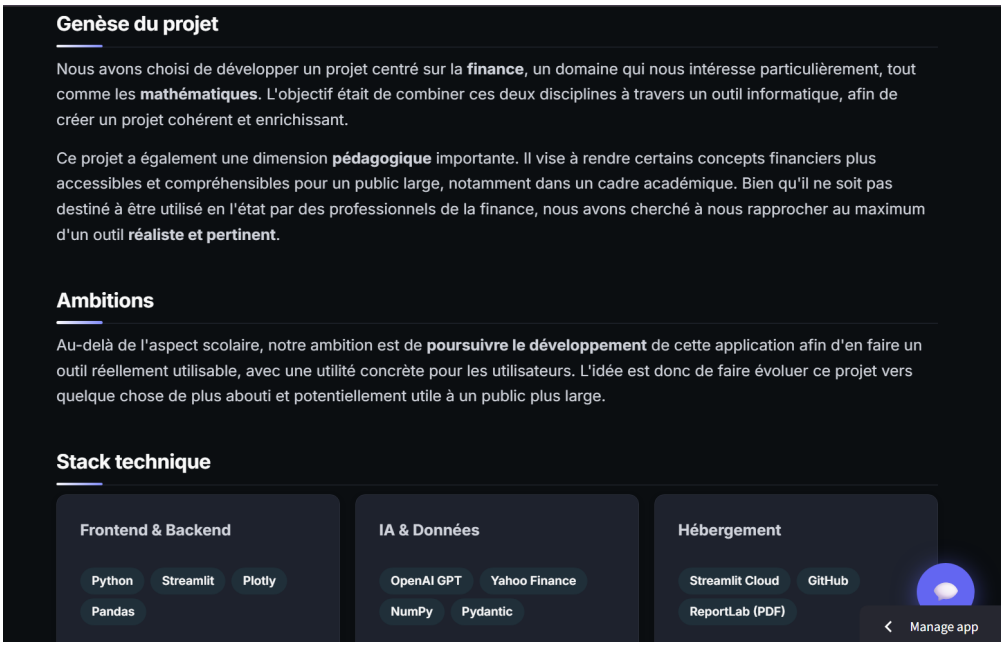


FIGURE 26 – Stack technique en trois piliers : Frontend/Backend, IA/Données, Hébergement

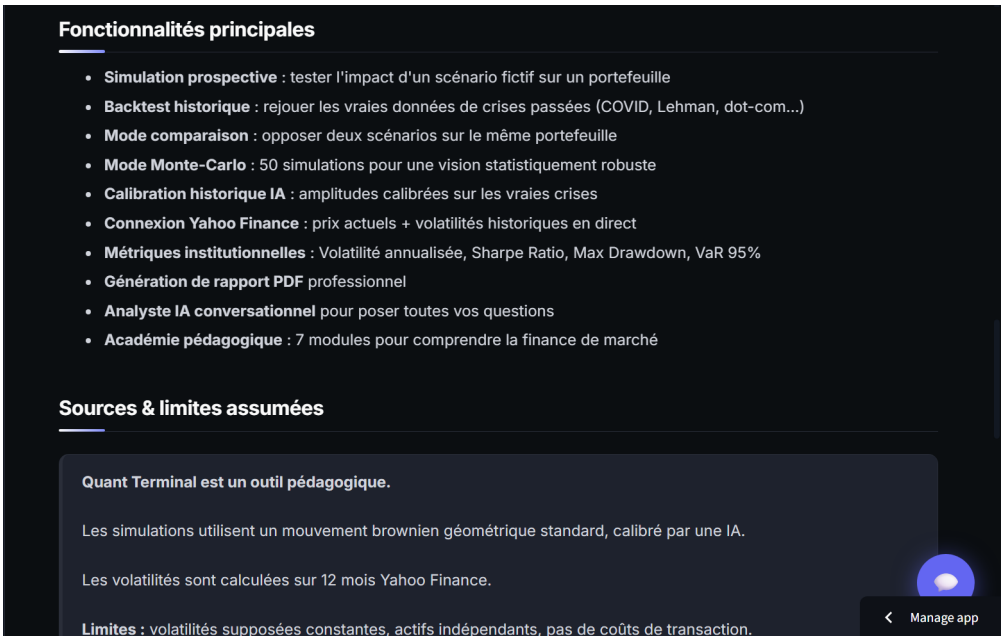
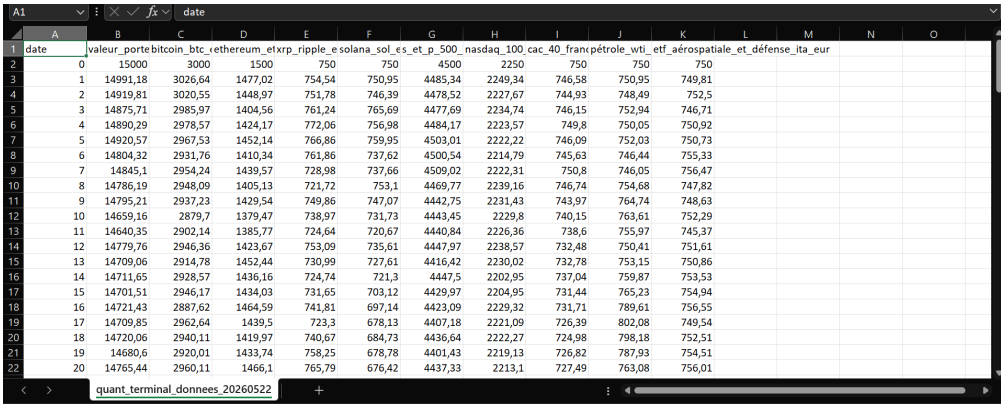


FIGURE 27 – Liste des fonctionnalités et section *Sources et limites assumées* : Quant Terminal est un outil pédagogique utilisant un GBM avec volatilités supposées constantes et actifs indépendants



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	date	valeur_porte	bitcoin_btc_	ethereum_et	xrp_ripple_e	solana_sol_e	s_et_p_500_	nasdaq_100_	cac_40_franc	pétrole_wti_	etf_aérospatiale_et_défense_ita_eur				
2	0	15000	3000	1500	750	750	4500	2250	750	750	750				
3	1	14991,18	3026,64	1477,02	754,54	750,95	4485,34	2249,34	746,58	750,95	749,81				
4	2	14919,81	3020,55	1448,97	751,78	746,39	4478,52	2227,67	744,93	748,49	752,5				
5	3	14875,71	2985,97	1404,56	761,24	765,69	4477,69	2234,74	746,15	752,94	746,71				
6	4	14890,29	2978,57	1424,17	772,06	756,98	4484,17	2223,57	749,8	750,05	750,92				
7	5	14920,57	2967,53	1452,14	766,86	759,95	4503,01	2222,22	746,09	752,03	750,73				
8	6	14804,32	2931,76	1410,34	761,86	737,62	4500,54	2214,79	745,63	746,44	755,33				
9	7	14845,1	2954,24	1439,57	728,98	737,66	4509,02	2222,31	750,8	746,05	756,47				
10	8	14786,19	2948,09	1405,13	721,72	753,1	4469,77	2239,16	746,74	754,68	747,82				
11	9	14795,21	2937,23	1429,54	749,86	747,07	4442,75	2231,43	743,97	764,74	748,63				
12	10	14659,16	2879,7	1379,47	738,97	731,73	4443,45	2229,8	740,15	763,61	752,29				
13	11	14640,35	2902,14	1385,77	724,64	720,67	4440,84	2226,36	738,6	755,97	745,37				
14	12	14779,76	2946,36	1423,67	753,09	735,61	4447,97	2238,57	732,48	750,41	751,61				
15	13	14709,06	2914,78	1452,44	730,99	727,61	4416,42	2230,02	732,78	753,15	750,86				
16	14	14711,65	2928,57	1436,16	724,74	721,3	4447,5	2202,95	737,04	759,87	753,53				
17	15	14701,51	2946,17	1434,03	731,65	703,12	4429,97	2204,95	731,44	765,23	754,94				
18	16	14721,43	2887,62	1464,59	741,81	697,14	4423,09	2229,32	731,71	789,61	756,55				
19	17	14709,85	2962,64	1439,5	723,3	678,13	4407,18	2221,09	726,39	802,08	749,54				
20	18	14720,06	2940,11	1419,97	740,67	684,73	4436,64	2222,27	724,98	798,18	752,51				
21	19	14680,6	2920,01	1433,74	758,25	678,78	4401,43	2219,13	726,82	787,93	754,51				
22	20	14765,44	2960,11	1466,1	765,79	676,42	4437,33	2213,1	727,49	763,08	756,01				

quant_terminal_donnees_20260522

FIGURE 28 – Export CSV dans Excel : une colonne par actif, une ligne par jour simulé

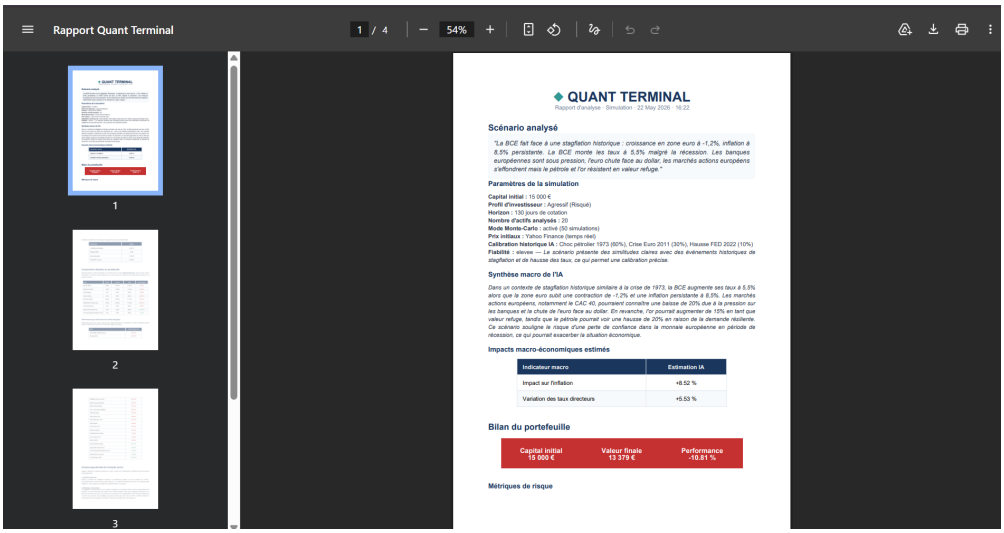


FIGURE 29 – Rapport PDF généré (4 pages) : scénario, paramètres, synthèse macro de l'IA, impacts par actif et métriques de risque

Annexe C : Manuel utilisateur

Prérequis pour lancer en local

- Python 3.10 ou supérieur installé sur la machine
- Un fichier `.env` à la racine du projet contenant : `OPENAI_API_KEY=sk-...`
- Dépendances installées via : `pip install -r requirements.txt`
- Lancer l'application avec : `streamlit run app.py`

L'application est accessible directement en ligne (sans installation) à : `https://quant-terminal.streamlit.app`

Guide des onglets

- **Dashboard** : résultats principaux, valeur finale, analyse IA, graphiques d'évolution, Monte-Carlo par catégorie, métriques de risque, alertes
- **Portefeuille** : détail par actif avec performance individuelle et donut d'allocation
- **Comparaison** : disponible si 2 ou 3 scénarios sont lancés, graphique d'évolution comparée et tableau de métriques côte à côte
- **Historique** : 10 dernières simulations sauvegardées automatiquement, persistantes entre les sessions
- **Académie** : 7 modules pédagogiques sur la finance de marché, conçus pour être lus dans l'ordre
- **À propos** : présentation du projet, de l'équipe et de la stack technique
- **Chatbot IA** : accessible via la bulle violette en bas à droite, pour poser des questions financières en langage naturel